

SHOP OS

Simple Shop Management & Fast Billing Platform

Complete Product + UX/UI + MERN Technical Documentation

Cursor-ready master specification for building the product from zero to production.

Product principle:

“Dukaan chalao. Data entry nahi.”

Version 1.0 • August 2026

1. Product Vision

SHOP OS is a simple digital counter and shop diary for small Indian retailers. It records sales, payments, stock, customers, udhaar, expenses and monthly performance without forcing the owner to understand accounting software.

Core promise:

**“Customer aaya → samaan select karo → payment choose karo → sale complete.
Baaki hisaab app sambhale.”**

Target: 1-5 employee retail shops: kirana/general stores, stationery, cosmetics, hardware, electrical, bakery, gift and small accessory shops.

Positioning:

Do not build “another accounting app.” Build the simplest and fastest digital shop counter.

2. Product Principles

Principle	Requirement
Zero forced setup	Never require 100-150 products before first sale.
Fast billing	Common sale should take only a few obvious actions.
Progressive disclosure	Basic fields first; advanced/GST fields hidden.
Cash first-class	Cash, UPI, card and credit are valid payment paths.
Sale ≠ payment	A sale can be unpaid, partial or fully paid.
No fake intelligence	Profit/predictions only use available reliable data.
Offline core	Billing/catalogue should survive temporary internet loss.
Human language	“Aaj ki bikri”, “Udhaar baaki”, “Kharcha” instead of jargon.
Privacy	Cashier should not see cost/profit unless permitted.

3. Modules and Data Models

Use 14 primary business models in the MVP/V1, plus supporting technical collections.

Model	Purpose
User	Login and identity.
Shop	Business/tenant.
Membership	User ↔ Shop + role/permissions.
Product	Item sold by the shop.
ProductCategory	Grouping.
Sale	Commercial transaction.
SaleItem	Products inside a sale.
Payment	Cash/UPI/card/credit/partial payments.
Customer	Optional customer profile.
InventoryTransaction	Auditable stock movements.
Expense	Shop operating expenses.
Supplier	Vendor.
Purchase	Stock purchase.
PurchaseItem	Items in purchase.

Model	Purpose
CashSession	Expected vs actual cash.
Notification	Alerts/reminders.
AuditLog	Sensitive action trail.
Subscription	Plans/billing, later.

Supporting technical collections: sessions/refresh tokens, OTP verification, sync queue, idempotency keys and integration configuration.

4. UX/UI System

The primary competitive advantage is usability. A person uncomfortable with smartphones should still be able to make a sale without training.

- Mobile: Home, New Bill, Products, Customers, More. New Bill is always prominent.
- Desktop: left navigation with a counter/POS layout. Support keyboard shortcuts.
- Use large tap targets, readable text, clear icons + words, high contrast and obvious primary actions.
- Do not force customer details for normal sales. Default to Walk-in Customer.
- Do not show GST, SKU, HSN, purchase cost or other advanced fields unless enabled/needed.
- Always provide loading, empty, error and success states.

Golden rule:

Every primary screen should answer: “Ab mujhe kya karna hai?”

5. Core Screens

Home

Good Morning ☀

Aaj ki bikri
₹8,450
32 sales

[+ NAYA BILL]

Is mahine
₹2,48,500 sale
₹48,200 estimated profit*

⚠ 12 items low
☐ ₹8,450 udhaar baaki
☐ ₹2,000 expenses

*Only when reliable cost data exists.

New Bill

← New Bill

[☐ Product search...]

☐ Frequently Sold
[Maggi] [Milk] [Bread] [Biscuit]

Cart
Maggi ×2 ₹28
Milk ×1 ₹30
Biscuit ×2 ₹40

TOTAL ₹98

[CASH] [UPI] [UDHAR]

[MORE PAYMENT OPTIONS]

[COMPLETE SALE]

Sale Complete

SALE COMPLETED ✓

₹450

Payment: Cash

[WhatsApp Bill]

[Print]

[Don't Send]

[NEW BILL]

Products: search → frequently sold → categories → low stock → Add Product → import.

Customers: name/phone optional; show current due, last purchase, total purchases and payment history.

Reports: Today / This Month / Custom range with sales, payment split, top products, expenses, udhaar and estimated profit.

6. Product Onboarding — Solve the 100-150 Item Problem

Never tell a new shopkeeper to manually enter all products before first use.

A. Add While Selling

Search: Parle-G

No product found

[+ Add "Parle-G"]

Price: ₹10

Track stock? ON

[ADD & SELL]

B. Generic/Quick Items

For ₹1/₹2 toffees and chocolates allow simple items such as ₹1 Toffee, ₹2 Chocolate, ₹5 Biscuit. No brand, SKU, barcode or GST is required.

C. Starter Catalogue

Selecting Kirana can suggest common categories/products. Suggested stock must start at zero.

D. Excel/CSV Import

Import name, sellingPrice, purchasePrice(optional), openingStock(optional), unit, barcode, category. Preview validation errors before commit.

E. Supplier Invoice Scan

Future AI: extract products/quantities from supplier invoice, show review, then user confirms stock mutation.

F. Shelf Photo

Experimental only. AI detection must never silently create inventory; user must review.

Track stock toggle:

If false, sales and quantities are recorded but inventory quantity is not maintained.

7. Billing + Payment Logic

Core rule: sale, invoice and payment are separate concepts.

Method	MVP behavior	Later
Cash	Received amount + automatic change.	Cash drawer reconciliation.
UPI	Manual "Payment Received" confirmation.	Integrated UPI/payment gateway.
Card	Record payment.	Terminal integration.
Credit/Udhaar	Customer + outstanding balance.	Automated reminders.
Split	V1: multiple payments on one sale.	Deeper reconciliation.

Cash example

Bill total ₹128
 Customer gives ₹200
 Received ₹200
 Change ₹72
 Sale total remains ₹128

UPI example

In MVP the shopkeeper taps UPI and confirms receipt. Later an integrated provider can make confirmation server-authoritative.

No bill requested:

The sale is still saved. "Don't Send" means do not share/print the invoice; it never deletes the sale.

Partial payment:

Total ₹2,000
 Received ₹1,000
 Due ₹1,000

 Later receive ₹1,000
 → create Payment
 → Due becomes ₹0
 → DO NOT create another Sale

8. Inventory

Do not use a blind mutable stock number as the only source of truth. Maintain an auditable InventoryTransaction ledger.

Transaction types: OPENING_STOCK, PURCHASE_IN, SALE_OUT, SALE_RETURN_IN, PURCHASE_RETURN_OUT, MANUAL_ADJUSTMENT_IN, MANUAL_ADJUSTMENT_OUT, DAMAGE_OUT, STOCK_COUNT_ADJUSTMENT.

```

availableStock =
opening
+ purchaseIn + returnIn + adjustmentIn
- saleOut - purchaseReturn - damageOut - adjustmentOut
  
```

Low stock triggers when `availableStock <= minimumStock` and `trackStock=true`.

Default strict mode prevents negative stock; optional setting can allow it with a warning. Never silently reset negative stock to zero.

9. Customers + Udhaar

Walk-in Customer is the default. Customer creation should be optional for ordinary sales.

Sale total ₹1,000
 Received ₹0
 Credit due ₹1,000

 Later payment:
 Payment ₹1,000
 Credit due ₹0

 Later payment is NOT another sale.

Customer page: due amount, last purchase, total purchases, payment history and one-tap reminder.

10. Expenses + Cash Reconciliation

Expense categories: Rent, Electricity, Transport, Tea/Food, Salary/Wages, Repairs, Other.

```
Cash sales ₹12,500
Cash received on credit ₹1,000
Cash expenses -₹2,000
Cash refunds -₹500
-----
Expected cash ₹11,000

Actual drawer cash ₹10,800
Difference -₹200
```

Use friendly labels “Expected cash” and “Actual cash”.

11. Reports

Monthly sales/revenue = sum of finalized Sale.total for the selected date range; exclude cancelled/voided sales.

Payment breakdown = sum Payment.amount by method. Credit can be revenue even when cash received is zero.

Estimated profit = item revenue – reliable item cost. If cost coverage is incomplete, label it “Estimated profit” and show coverage; never invent missing costs.

- Today / Week / Month / Custom range
- Sales by Cash/UPI/Card/Credit
- Top products
- Low stock
- Udhaar outstanding
- Expenses
- Estimated profit

12. Roles + Privacy

Role	Access
OWNER	Everything: settings, cost/profit, reports, staff, subscriptions.
MANAGER	Sales, inventory, customers and configured reports.
CASHIER	Sales and necessary product/payment data; no cost/profit by default.
INVENTORY_STAFF	Products, stock counts and purchases; no financial reports unless permitted.

Permission examples: sale.create, sale.refund, product.create, product.edit, inventory.adjust, purchase.create, customer.view, customer.credit, report.view, cost.view, expense.create, shop.settings.

All records are shop-scoped. Server must verify membership; never trust client-provided shopId.

13. Offline-First + Sync

```
ONLINE
React → API → MongoDB

OFFLINE
React → IndexedDB → Sync Queue

RECONNECT
Queue → API + Idempotency Key
→ validate → MongoDB
```

→ mark synced

- Sale creation must be idempotent so retries cannot create duplicates.
- Stock mutations are transaction/event based, not blind overwrites.
- Product edits use versions/timestamps.
- If two devices sell the last item, server reconciliation must preserve an auditable result.
- Show sync status without blocking ordinary selling.

14. MERN Architecture

```
Client
React + TypeScript + Vite + PWA
|
v
Node.js + Express + TypeScript
|
MongoDB + Mongoose
|
Redis + BullMQ (jobs)
|
S3-compatible storage / integrations
|
WhatsApp / Payment / AI-OCR
```

Recommended stack: React/TypeScript/Vite, Tailwind, TanStack Query, IndexedDB, Node/Express/TypeScript, MongoDB/Mongoose, Zod/shared schemas, secure HTTP-only session/refresh strategy, Redis/BullMQ, S3-compatible storage.

Folder structure

```
apps/
web/src/
features/{auth,billing,products,customers,inventory,purchases,expenses,reports,settings}
components/ pages/ hooks/ lib/ offline/
api/src/
modules/{auth,shops,memberships,products,sales,payments,customers,inventory,purchases,expenses,reports,notifications}
middleware/ jobs/ lib/ app.ts
packages/shared/{schemas,types,constants}
```

15. MongoDB Schemas

User

```
User {
  _id, phone, email?, name?, avatarUrl?,
  isPhoneVerified, createdAt, updatedAt
}
```

Shop

```
Shop {
  _id, name, businessType?, ownerId,
  currency:"INR",
  settings:{simpleMode,allowNegativeStock,defaultPaymentMethod,
  gstEnabled,invoicePrefix?},
  createdAt, updatedAt
}
```

Membership

```
Membership {
  _id, shopId, userId, role, permissions[], status,
  createdAt, updatedAt
}
```

Product

```
Product {
  _id, shopId, name, normalizedName, categoryId?,
  sku?, barcode?, unit:"piece",
  sellingPrice, purchasePrice?, mrp?,
  trackStock, minStock?, active, createdAt, updatedAt
}
```

Sale / SaleItem

```
Sale {
  _id, shopId, invoiceNumber, customerId?,
  status:"DRAFT"|"COMPLETED"|"CANCELLED"|"REFUNDED",
  subtotal, discount, tax, total, amountPaid, amountDue,
  createdBy, completedAt, createdAt, updatedAt
}
```

```
SaleItem {
  _id, saleId, productId, productNameSnapshot,
  quantity, unitPrice, unitCostSnapshot?,
  discount, tax, lineTotal
}
```

Payment

```
Payment {
  _id, shopId, saleId?, customerId?,
  method:"CASH"|"UPI"|"CARD"|"CREDIT"|"BANK_TRANSFER",
  amount, status:"PENDING"|"CONFIRMED"|"FAILED"|"REFUNDED",
  reference?, receivedAt, createdBy, createdAt
}
```

Customer

```
Customer {
  _id, shopId, name, phone?, notes?, creditLimit?,
  active, createdAt, updatedAt
}
```

InventoryTransaction

```
InventoryTransaction {
  _id, shopId, productId, type, quantityDelta,
  sourceType, sourceId?, unitCost?, note?,
  createdBy, createdAt
}
```

Expense / Supplier / Purchase

```
Expense { _id, shopId, category, amount,
  paymentMethod:"CASH"|"UPI"|"BANK", note?, spentAt, createdBy }
```

```
Supplier { _id, shopId, name, phone?, address?, active }
```

```
Purchase { _id, shopId, supplierId?, status, total, purchasedAt, createdBy }
```

```
PurchaseItem { _id, purchaseId, productId, productNameSnapshot,
  quantity, unitCost, lineTotal }
```

CashSession / AuditLog

```
CashSession {
  _id, shopId, openedBy, openingCash, expectedCash,
  actualCash?, difference?, openedAt, closedAt?
}
```

```
AuditLog {
  _id, shopId, actorUserId, action, entityType, entityId,
  metadata, createdAt
}
```

Indexes: Product {shopId,normalizedName}, optional unique {shopId,barcode}; Sale {shopId,completedAt} and unique {shopId,invoiceNumber}; Payment {shopId,receivedAt}; InventoryTransaction {shopId,productId,createdAt}; Customer {shopId,phone}; Membership unique {shopId,userId}.

16. REST API

Area	Endpoint	Purpose
Auth	POST /api/auth/start	Start login/OTP
Auth	POST /api/auth/verify	Verify OTP/session
Shop	POST /api/shops	Create shop
Shop	GET /api/shops/:shopId	Get shop
Products	GET /api/shops/:shopId/products	Search/list
Products	POST /api/shops/:shopId/products	Create
Products	POST /api/shops/:shopId/products/import	Bulk import
Sales	POST /api/shops/:shopId/sales	Create/complete sale
Sales	GET /api/shops/:shopId/sales	List sales
Sales	GET /api/sales/:id	Detail
Payments	POST /api/sales/:id/payments	Record payment
Payments	POST /api/payments/:id/refund	Refund
Customers	GET /api/shops/:shopId/customers	Search/list
Customers	POST /api/shops/:shopId/customers	Create
Credit	GET /api/shops/:shopId/credit	Outstanding credit
Credit	POST /api/customers/:id/payments	Collect udhaar
Inventory	GET /api/shops/:shopId/inventory	Inventory
Inventory	POST /api/shops/:shopId/inventory/adjust	Adjust
Expenses	POST /api/shops/:shopId/expenses	Record expense
Reports	GET /api/shops/:shopId/reports/summary	Dashboard
Reports	GET /api/shops/:shopId/reports/sales	Sales report
Staff	POST /api/shops/:shopId/members	Add member

API rules: server recomputes totals; consistent errors {code,message,fieldErrors?}; pagination; server timestamps; idempotency for financial mutations; server-side authorization.

17. All Important Edge Cases

Case	Expected behavior
₹200 cash for ₹128	Received ₹200, change ₹72; sale total ₹128.
Exact cash	Change ₹0.
No bill wanted	Save sale; do not share.
WhatsApp fails	Sale remains completed; allow retry.
Partial payment	Keep amountDue; do not create extra sale.
Later credit payment	Create Payment only; revenue unchanged.
Product not found	Quick Add → Add & Sell.
₹1 chocolate	Generic quick item; no forced barcode/GST.
Stock tracking off	Record sale but do not maintain quantity.
Duplicate product	Suggest existing; explicit create or future merge.
Refund	Create reversal/refund; never hard-delete financial history.
Price changes	SaleItem keeps price snapshot.
Product renamed	SaleItem keeps productNameSnapshot.

Case	Expected behavior
Missing cost	No precise profit; show estimated/partial.
Offline retry	Idempotency prevents duplicate sale.
Two devices	Server reconciliation + transaction ledger.
Cross-shop request	Reject unless membership authorizes shop.

18. Security + Reliability

- Tenant isolation on every query.
- Role-based authorization on sensitive endpoints.
- Secure HTTP-only authentication/session approach.
- Rate-limit auth and public endpoints.
- Validate every request server-side.
- Never log OTPs, tokens or payment secrets.
- Audit refunds, stock adjustments, price changes and role changes.
- Back up MongoDB and test restore procedures.
- Encrypt data in transit and at rest.
- Never expose cost/profit to unauthorized staff.

19. WhatsApp + Notifications

Sale completion must not wait for messaging delivery. Queue sharing jobs.

```

Sale completed
↓
Invoice payload
↓
Queue
↓
WhatsApp provider
↓
Delivered / Failed
↓
Notification status

```

Events: LOW_STOCK_TRIGGERED, CREDIT_PAYMENT_DUE, DAILY_SUMMARY_READY, MONTHLY_SUMMARY_READY, IMPORT_COMPLETED, INVOICE_SHARE_FAILED. Keep provider-specific code behind an integration interface.

20. AI Features

Feature	Phase	Safety rule
Supplier invoice extraction	V2	Review required before stock changes.
Duplicate matching	V2	Suggest only; never silent merge.
Purchase suggestion	V2	Use recent sales + stock; show as suggestion.
Natural-language summary	V2	Answers grounded only in shop data.
Shelf photo recognition	Experimental	Human confirmation mandatory.

AI should reduce data entry, not create a chatbot for decoration.

21. Testing

Unit tests: totals, change, payment allocation, stock deltas, profit coverage, permissions.

Integration tests: cash sale updates sale/payment/stock; credit collection does not create revenue; refund reverses impact; offline retry is idempotent; cross-shop access denied.

E2E: new user → shop → quick product → cash sale; CSV import; WhatsApp failure/retry; cashier cannot view cost.

UX acceptance: first-time user can create a first sale without documentation; common sale is fast; no irrelevant required fields; mobile and desktop both work.

22. Roadmap

Release	Scope
MVP	Auth, shop, Quick Add, generic items, fast billing, Cash/UPI/Credit, invoice, customers, basic inventory, daily/monthly sales
V1	Expenses, udhaar collection, low stock, staff roles, supplier/purchases, barcode, CSV import, cash reconciliation, offline har
V1.5	Split payment, advanced reports, product merge, invoice customization, notifications.
V2	AI invoice scan, purchase suggestions, duplicate detection, natural-language reports, optional payment integrations.
V3	Multi-shop analytics, POS hardware, deeper GST/accounting integrations, supplier ecosystem.

23. Monetization

Plan	Suggested scope
Free	1 shop, basic billing/products/customers/credit, limited advanced reports.
Starter ~₹99/mo	Unlimited bills/products, WhatsApp, inventory, reports.
Pro ~₹249/mo	AI, smart stock suggestions, employee roles, advanced reports.
Shop+ ~₹499/mo	Multiple shops/counters, advanced permissions/analytics.

Pricing must be validated with actual users. Compete on simplicity and speed, not only low price. A strong acquisition feature is migration from Excel/CSV so existing shop owners do not have to re-enter their catalogue.

24. Cursor Master Prompt

You are the lead engineer for SHOP OS.

STACK

React + TypeScript + Vite + PWA
Tailwind CSS
Node.js + Express + TypeScript
MongoDB + Mongoose
Redis + BullMQ where needed
IndexedDB offline queue
REST API
Shared TypeScript schemas/types

PRODUCT RULES

1. Small-shop users, including low-tech smartphone users.
2. Never force 100-150 products before first sale.
3. Add While Selling, Quick Items and Walk-in Customer.
4. Cash, UPI and Credit are first-class.
5. Sale, invoice and payment are separate.
6. Credit collection never creates another sale.
7. Inventory is an auditable transaction ledger.
8. Historical sale items keep snapshots.
9. Every shop API checks server-side membership.
10. Simple mode hides GST/accounting complexity.

ENGINEERING

- Strict TypeScript.
- Modular features.
- Server validation.
- Typed API contracts.
- Unit/integration/E2E tests.

- Idempotency for financial mutations.
- No hard deletion of financial records.
- Loading/empty/error/success states.
- Mobile + desktop UX.
- Do not use mock data in production paths.

Before coding a phase, return: architecture, files, schemas, APIs, tests, risks. Implement only that phase. After coding, run build/lint/tests and report results.

25. Cursor Phase Prompts

Phase 0 — Foundation

Create the monorepo, React/Vite/PWA frontend, Node/Express API, shared schemas/types, environment validation, logging, errors, testing, linting and formatting. Do not build business features.

Phase 1 — Auth/Shop/Membership

Implement User, Shop, Membership, secure authentication/session, shop creation and role authorization. Test cross-shop denial.

Phase 2 — Products

Implement Product/Category, simple UI, Quick Add, Add & Sell, generic ₹1/₹2 items, trackStock, search, favorites and duplicate suggestions.

Phase 3 — Billing

Implement Sale/SaleItem and New Bill for mobile/desktop. Product search, quick buttons, quantity, server totals, completion and keyboard shortcuts.

Phase 4 — Payments

Implement Payment for Cash/UPI/Credit/Card, cash change, partial payments and idempotency. Prove credit collection cannot create another sale.

Phase 5 — Inventory

Implement InventoryTransaction, stock views, opening stock, adjustments and low-stock rules. Test stock integrity.

Phase 6 — Customers/Udhaar

Implement Customer, outstanding balance, payment collection, history, reminders and Walk-in Customer.

Phase 7 — Reports/Expenses

Implement Expense and Today/Month reports, payment breakdown, top products, udhaar, expenses and estimated profit with cost coverage.

Phase 8 — Invoice/Sharing

Generate invoices and print/share flows. Sharing failure must never rollback a sale. Add optional GST settings.

Phase 9 — Staff/Privacy

Implement Owner/Manager/Cashier/Inventory roles, permissions and audit logs. Hide purchase cost/profit from unauthorized staff.

Phase 10 — Migration

Implement CSV/Excel import with preview, validation, duplicate suggestions and rollback-safe commit.

Phase 11 — Offline

Implement IndexedDB cache and mutation queue, idempotency keys, sync UI and tests for reconnect/duplicate prevention.

Phase 12 — Supplier/Purchase

Implement Supplier, Purchase, PurchaseItem and inventory updates through transactions.

Phase 13 — AI

Add feature-flagged invoice extraction and duplicate suggestions. Human confirmation required before mutations.

Phase 14 — Hardening

Security audit, authorization audit, indexes, race conditions, offline sync, backups, rate limits, accessibility and full E2E.

26. Final Acceptance Checklist

- First-time user can make a first sale without entering all products.
- Add While Selling works. ₹1/₹2 generic items work.
- Walk-in customer works.
- Cash change works. UPI confirmation works. Credit and partial payments work.
- Later udhaar collection never counts as new revenue.
- WhatsApp/print is optional and cannot rollback sales.
- Stock decreases only when sale is finalized.
- Refund reverses impact without deleting history.
- Monthly sales are accurate. Profit is marked estimated when cost is incomplete.
- Low-stock alerts respect trackStock.
- Cashier cannot see restricted financial data.
- Offline sales sync without duplication.
- Historical invoices stay unchanged after product edits.
- Every shop-scoped API authorizes membership.
- Critical financial actions are auditable.
- Mobile UI is usable by a low-tech user.
- Desktop billing supports keyboard-first operation.

North-star UX test:

Give the app to a shopkeeper who has never seen it and say only: “Isme ek customer ka bill banao.” If they need an explanation for basic billing, simplify the UI before adding more features.

END OF SPECIFICATION